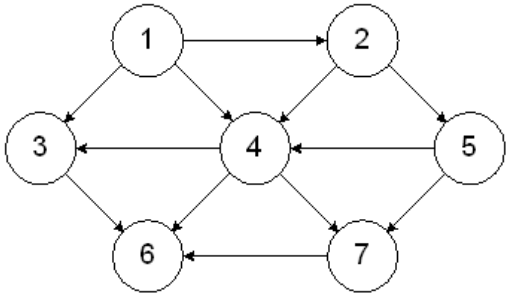
EJERCICIO 1

Añade al TAD Grafo binario una función que permita calcular el grado saliente de un nodo (*out-degree*), es decir, el número de relaciones con otros nodos del grafo. Prográmalo para ambas implementaciones: matriz y lista de adyacencia.

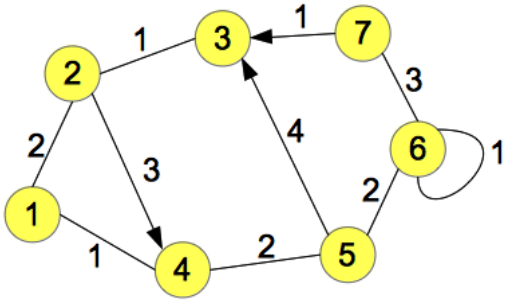


En este ejemplo el out-degree del nodo 5 es 2, mientras que el del 4 es 3

```
function outdegree(g:edgeWeightedDigraph; node:string): integer;
// Out-degree incluyendo autoenlaces. Sin autoenlaces i<>node_index en el for
var i,node_index:integer;
begin
   outdegree:= 0;
   node_index:= get_index(g,node);
   for i:= 1 to MAX_NODES do begin
     if g.matrix[node_index,i]<>'inf' then outdegree:= outdegree+1
   end;
end;
```

EJERCICIO 2

Completa el TAD Grafo valorado con otra función que compute el peso de cada nodo en función de los pesos de sus enlaces salientes y devuelva dichos valores en una lista donde habrá un peso para cada nodo. Prográmalo para ambas implementaciones: matriz y lista de adyacencia



En este ejemplo se retornaría la lista siguiente: [3,6,1,3,4,6,4] donde 3 es el peso del out-degree del nodo 1, 6 el del nodo 2 y así sucesivamente.

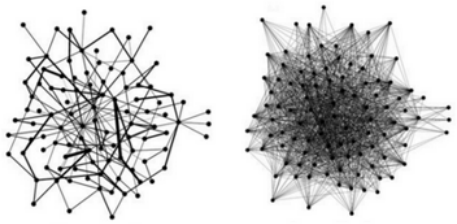
SOLUCIÓN:

```
procedure get_weights_list(g:edgeWeightedDigraph; var lista:tlista);
var i,j,contador:integer;
begin
  uListaEnteros.initialize(lista);
  for i:=1 to g.num_nodes do begin
    contador:= 0;
    for j:=1 to g.num_nodes do
      if g.matrix[i,j]<>'inf' then contador:= contador + StrToInt(g.matrix[i,j]);
    insert_at_end(lista,contador);
  end;
end;
```

Nota: hay que añadir al TAD grafo la siguiente línea: uses uListaEnteros, sysutils;

EJERCICIO 3

Añade al TAD Grafo binario una función que permita calcular su densidad, sabiendo que la densidad de un grafo binario es un valor real entre 0 y 1 que representa el número de arcos que tiene el grafo dividido entre el número potencial de arcos que podría llegar a tener. No deben tenerse en cuenta los auto-enlaces. Prográmalo para ambas implementaciones: matriz y lista de adyacencia.



Grafo disperso Grafo denso

SOLUCIÓN:

Teniendo en cuenta que:

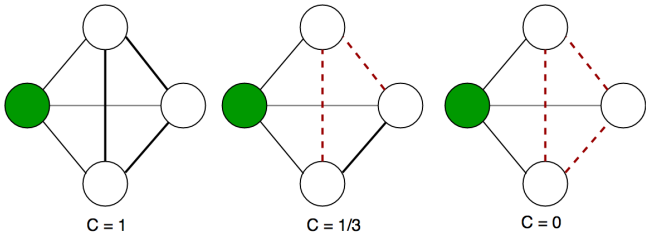
- *Densidad = número de arcos existentes / número de arcos posibles*
- *Número de arcos posibles en un grafo binario = n·(n-1)/2*

Asumiendo pues que el TAD GrafoBinario está implementado internamente con una matriz de adyacencia en la cual hay un valor 1 tanto en M[A,B] como en M[B,A] si existe un enlace entre los nodos A y B, y un cero en caso contrario, la función densidad se implementaría así:

```
function density(g:edgeWeightedDigraph): real;
var i,j:integer;
begin
  density:=0;
  if g.num_nodes>1 then begin // Un grafo con menos de 2 nodos, densidad = 0
    for i:=1 to g.num_nodes do
      for j:=i+1 to g.num_nodes do
        if g.matrix[i,j]<>0 then density:= density + 1;
    density:= 2*density / (g.num_nodes*(g.num_nodes-1))
  end;
end;
```

EJERCICIO 4

Añade al TAD Grafo binario una función que permita calcular el coeficiente de agrupamiento (o *clustering*) de un nodo. Dicho coeficiente ("C" en la Figura) se calcula como la densidad del grafo que forman los nodos de su vecindario. Prográmalo para ambas implementaciones: matriz y lista de adyacencia y calcula la complejidad de tus soluciones.



SOLUCIÓN:

Para resolver este ejercicio creamos un subgrafo con el vecindario del nodo cuyo coeficiente de clustering queremos calcular y posteriormente calculamos su densidad utilizando la función implementada en el ejercicio 3. Al igual que el anterior, asume que el TAD `GrafoBinario` está implementado internamente con una matriz de adyacencia con valores 1 y 0.

```
function clustering_coef(g:edgeWeightedDigraph; node:string): real;
var
   i,j,indice_origen:integer;
   vecindario: edgeWeightedDigraph;
   origen, destino, peso: string;
begin
  initialize(vecindario);
  indice_origen:= get_index(g,node);
  for i:= 1 to g.num_nodes do  // Añade todos los nodos vecinos al vecindario
     if (i<>indice_origen) and (g.matrix[indice_origen,i]<>0) then
       insert_node(vecindario,g.node_labels[i]);
  for i:= 1 to vecindario.num_nodes do  // Añade todos los enlaces al vecindario
    for j:= 1 to g.num_nodes do begin
      origen:= vecindario.node_labels[i];
      destino:= g.node_labels[j];
      peso:= g.matrix[get_index(g,origen),j];
      // Si el nodo del grafo original (destino) pertenece al vecindario y tiene
      // un enlace con el nodo actual del vecindario (origen), incluir esa
      // relación en el grafo vecindario
      if (node_in_graph(vecindario,destino)) and (peso<>0) then
        insert_edge(vecindario,origen,destino,peso);
    end;
  // El coef. clustering de un nodo es la densidad de su grafo vecindario
  clustering_coef:= density(vecindario);
end;
```

EJERCICIO 5

Escribe un algoritmo para determinar si un grafo dirigido implementado con una matriz de adyacencia contiene algún autoenlace, es decir, un arco que tenga origen y destino en el mismo nodo. *Complejidad de tiempo esperada: O(n). Espacio auxiliar esperado: O(1)*

Reescribe tu algoritmo ahora para un grafo implementado con listas de adyacencia y calcula la complejidad de tu solución.

SOLUCIÓN:

```
function has_self_loops(g: edgeWeightedDigraph): boolean;
var i:integer;
begin
  has_self_loops:= false;
  i:=0;
  while (not has_self_loops) and (i<g.num_nodes) do begin
    i:= i+1;
    has_self_loops:= g.matrix[i,i]<>'inf';
  end;
end;
```

La complejidad es O(n) pues en el peor de los casos no tendrá autoenlaces y recorrerá todos los nodos del grafo.

EJERCICIO 6

Implementa un procedimiento externo (no pertenece al TAD árbol ni al TAD grafo) que genere un grafo binario no dirigido a partir de un árbol binario de búsqueda. El grafo contendrá todos los elementos del árbol, de tal modo que los enlaces padre-hijo del árbol serán arcos en el grafo. Calcula la complejidad de tu solución.

SOLUCIÓN:

Asumimos que además de las esperadas `initialize` e `is_empty`, el TAD `tBinarySearchTree` implementa las siguientes tres operaciones:

- *`function get_info(a:tBinarySearchTree): integer;`*
- *`function get_hi(a:tBinarySearchTree): tBinarySearchTree;`*
- *`function get_hd(a:tBinarySearchTree): tBinarySearchTree;`*

También asumimos que el TAD `BinaryGraph` implementa las operaciones `initialize`, `insert_node` e `insert_edge`. Si esto es así, el procedimiento será:

```
procedure BST2Graph(a: tBinarySearchTree; var g: tBinaryGraph);
var hi,hd: tBinarySearchTree;
    info: string;
begin
  if not is_empty(a) then begin
    info:= intToStr(get_info(a));
    if not node_in_graph(g,info) then insert_node(g,info));
    hi:= get_hi(a);
    hd:= get_hd(a);
    if hi <> nil then begin
       insert_node(g,intToStr(get_info(hi)));
       insert_edge(g,intToStr(get_info(a)),intToStr(get_info(hi)));
       BST2Graph(hi,g);
    end;
    if hd <> nil then begin
       insert_node(g,intToStr(get_info(hd)));
       insert_edge(g,intToStr(get_info(a)),intToStr(get_info(hd)));
       BST2Graph(hd,g);
    end;
  end;
end;
```

El procedimiento deberá ser invocado previa inicialización del grafo.

En cuanto a la complejidad de nuestra solución, tenemos que es O(n) ya que forzosamente debe visitar todos los nodos del árbol.

EJERCICIO 7

Implementa un procedimiento que a partir de una lista de arcos genere un grafo ponderado dirigido. Cada arco en la lista tendrá origen, destino y peso. El procedimiento de creación y llenado del grafo creará los nodos que no existen cuando recibe un arco origen-destino. En caso de recibir un arco entre nodos entre los que ya exista un arco en el grafo, el peso del arco actual debe sumarse al del arco preexistente. Calcula la complejidad de tu solución.

SOLUCIÓN:

```
procedure graph_from_list(lista_arcos: tlistaarcos; var g: edgeWeightedDigraph);
var
    lista_arcos: tlistaarcos;
    arco: tarco;
    i, peso: integer;
begin
  uMatrixEdgeWeightedDigraph.initialize(g);
  for i:=1 to num_elems(lista_arcos) do begin
    arco:= first(lista_arcos);
    delete_at_begin(lista_arcos);
    insert_at_end(lista_arcos,arco);
    peso:= strtoint(arco.peso);
    if edge_in_graph(g, arco.origen, arco.destino) then begin
        peso:= peso + strtoint(edge_weight(g,arco.origen,arco.destino));
        delete_edge(g,arco.origen,arco.destino);
      end
    else begin
       if not node_in_graph(g,arco.origen) then insert_node(g,arco.origen);
       if not node_in_graph(g,arco.destino) then insert_node(g,arco.destino);
    end;
    insert_edge(g,arco.origen,arco.destino,inttostr(peso));
  end;
end;
```

Todo lo anterior suponiendo que hay un TAD `ListaArcos` con la siguiente interfaz:

```
unit uListaArcos;

interface
  type
    tarco=record
      origen, destino, peso: string;
    end;

    nodo = record
      info: tarco;
      sig: ^nodo;
    end;

    tlistaarcos = record
      first, last: ^nodo
    end;

  procedure initialize(var list: tlistaarcos);
  function is_empty(list: tlistaarcos): boolean;
  function first(list: tlistaarcos): tarco;
  function last(list: tlistaarcos): tarco;
  procedure insert_at_end(var list: tlistaarcos; x: tarco);
  procedure insert_at_begin(var list: tlistaarcos; x: tarco);
  procedure delete_at_begin(var list: tlistaarcos);
  function num_elems(list: tlistaarcos): integer;
```

La complejidad es O(n), siendo n el número de arcos de la lista.

EJERCICIO 8 (Reto)

Te proponemos un ejercicio con mayor nivel de dificultad que los anteriores, pero muy interesante para profundizar en el estudio de los grafos. Vamos a determinar si un nodo de un grafo está involucrado en bucles (caminos que comienzan y terminan en el mismo nodo) y para ello te pedimos que resuelvas los siguientes 3 problemas en orden creciente de complejidad:

8.1. Escribe una función que determine si un cierto nodo de un grafo dirigido está involucrado en un bucle de orden 2 (camino de longitud 2 que comienza y finaliza en A).

SOLUCIÓN:

```
function in_loop(g:edgeWeightedDigraph; node:string): boolean;
var i, pos_node:integer;
begin
  in_loop:= false;
  pos_node:= get_index(g,node);
  if pos_node <> -1 then begin
     i:=0;
     while (not in_loop) and (i<g.num_nodes) do begin
       i:= i+1;
       if i<>pos_node then
         in_loop:= (g.matrix[pos_node,i]<>'inf') and (g.matrix[i,pos_node]<>'inf')
     end;
  end;
end;
```

8.2. Escribe una función que determine si un cierto nodo de un grafo dirigido está involucrado en un bucle de cualquier orden. La función simplemente determina si existe tal bucle o no.

[La solución consiste en modificar alguno de los recorridos, bien en anchura o bien en profundidad, para adaptarse al problema y detectar bucles]

8.3. Escribe una función que determine si un cierto nodo de un grafo dirigido está involucrado en un bucle de orden k. La función retornará el valor k, si hay bucle, o -1 si no existe tal bucle.

[La solución consiste en modificar la solución anterior para según vamos detectando bucles ir registrando el nivel de profundidad k]